When Behavioral Redirects Help or Hurt Code Agents: A Controlled Proxy Study

Anonymous Author(s)
Affiliation
Address
email

Abstract

Long agent trajectories make it difficult to decide whether a lightweight intervention will redirect unproductive behavior or distract an agent that is already reasoning coherently. We study observable post-error structure in 143 failed code-agent trajectories from SWE-bench Verified. We organize failures into four operational categories and evaluate ten one-sentence behavioral scaffolds in a controlled, one-step next-action setting across nine models. Under oracle diagnostic context, the candidate-best edit and plan scaffolds gain +1.00 and +0.88 on a 0–3 regex proxy. Yet the aggregate matched gain falls from +0.50 to 0.00 when its type-lexical relevance component is removed, and a blinded LLM judge gives +0.14 on 37 paired responses with an interval containing zero. Negative transfer is more persistent: using same-instance controls, 10 of 12 off-diagonal scaffold–category pairs are negative on the full proxy and 11 of 12 are negative without relevance. These experiments do not establish software repair. They expose evaluator-sensitive apparent gains and motivate controllers that validate redirects and retain an abstention option.

1 Introduction

Code agents can navigate repositories, edit files, and run tests, but their multi-step trajectories also create a control problem: after an error becomes visible, should an external controller redirect the agent, ask it to reconsider, or abstain? A single recovery instruction applied uniformly cannot distinguish a repetitive tool-use loop from a sequence of diverse but logically incorrect edits. Both trajectories fail, yet they expose different opportunities for inference-time control.

This distinction matters because plausible guidance can be actively counterproductive. In our controlled study, a test-guided instruction applied to localization failures scores 0.87 on a 0–3 next-action proxy, below the 1.70 unscaffolded control. The instruction is reasonable in isolation but assumes a failure pattern that is absent in the target trajectory. This observation suggests that intervention quality depends not only on the instruction itself, but also on the observable structure of the failure to which it is applied.

We study this structure in 143 failed trajectories from SWE-bench Verified [Jimenez et al., 2024]. We use four operational categories—failed edit application, localization, patch logic, and plan-level failure—and describe their post-error behavior as repetitive retry, mislocalized effort, reasoning drift, and commitment, respectively. The implemented post-error tail ratio measures how much of a trajectory occurs after its first observable error; it does not by itself label every later action as semantically unproductive.

We then conduct a controlled one-step experiment that holds the trajectory prefix and diagnostic context fixed while varying a short behavioral scaffold. The experiment measures the quality of the model's proposed next action rather than end-to-end task resolution. Three regularities emerge. First,

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

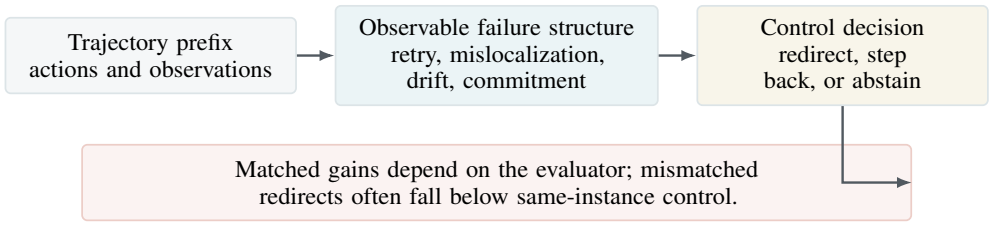

Figure 1: **Structure-aware control view.** Observable post-error behavior provides evidence for whether a lightweight behavioral redirect is appropriate. Our experiments expose an evaluator-sensitive structure–strategy interaction under controlled diagnostic context; they do not evaluate end-to-end repair.

matched effects differ by operational category on the full proxy. Second, these apparent gains are evaluator-sensitive and disappear in aggregate when the type-lexical relevance component is removed. Third, negative transfer persists across proxy specifications: a mismatched scaffold often scores below the same instance's control. Figure 1 summarizes the resulting control view.

Our contributions are:

- We give an operational four-category analysis of failed code-agent trajectories and report the distribution and post-error tail of each category (§3–§4).
- We analyze more than 1,400 next-action responses under matched, mismatched, held-out-selection, and cross-model conditions (§5).
- We identify a measurement boundary and a control risk: matched gains depend strongly on the lexical scoring component, while 10 of 12 paired cross-category transfers underperform control (§5.2).

2 Related Work

Agent self-correction and repair. Reflexion [Shinn et al., 2023], Self-Refine [Madaan et al., 2023], and self-debugging methods [Chen et al., 2023, 2025, Olausson et al., 2024] enable agents to recover from errors through self-generated feedback. RepairAgent [Bouzenia et al., 2025] and AgentCoder [Huang et al., 2024] extend this to multi-step autonomous repair. Chain-of-thought prompting [Wei et al., 2022], self-consistency [Wang et al., 2023], step-back abstraction [Zheng et al., 2024], and decomposed prompting [Khot et al., 2023] improve reasoning but do not specifically target recovery from prior errors. Tree-of-Thought [Yao et al., 2023], LATS [Zhou et al., 2024], Agent-R [Yuan et al., 2025], and GiGPO [Feng et al., 2025] explore recovery through branching, backtracking, or reward-trained policies. These methods typically evaluate a shared recovery mechanism in aggregate. We instead hold the mechanism lightweight and ask when its effect changes with the observable failure category.

Code agent failure analysis. SWE-bench [Jimenez et al., 2024] established the standard evaluation, and systems like SWE-agent [Yang et al., 2024], OpenHands [Wang et al., 2025], Agentless [Xia et al., 2025], AutoCodeRover [Zhang et al., 2024], CodeR [Chen et al., 2024], and MAGIS [Tao et al., 2024] represent the evolving agent landscape. AgentDebug [Zhu et al., 2025] and MAST [Cemri et al., 2025] categorize failures into stages or communication breakdowns. SWE-PRM [Gandhi et al., 2025] trains a process reward model with type-specific online feedback [Lightman et al., 2024, Uesato et al., 2022]. Nam et al. [2024] study how LLMs assist code understanding, complementing our focus on failure recovery. These works motivate failure-aware evaluation. Our narrower contribution is a controlled analysis connecting operational failure categories to the effect of short behavioral redirects.

Trace evidence and test-time control. DynaFix [Huang et al., 2025], TraceCoder [Huang et al., 2026], TraceFixer [Bouzenia et al., 2023], ARISE [Seddik and Fard, 2026], and RGFL [Sepidband et al., 2026] condition repair on execution traces, causal analysis, or program graphs. Test-time scaling and search methods allocate additional computation, whereas our study concerns a complementary control question: whether a low-cost behavioral redirect is supported by the observed trace. The

near-zero result for logic failures marks a setting in which richer execution evidence or search is likely necessary.

3 Operational Failure Categories

3.1 Data

We study 143 failed trajectories from the public O1-agent release on SWE-bench Verified [Jimenez et al., 2024], whose 500 tasks are drawn from 12 Python repositories.[1] Gold patches are used for retrospective categorization and evaluation; the analysis therefore concerns failed trajectories rather than leaderboard resolve rates. Each trajectory records a sequence of agent actions (file navigation, code edits, command executions) interleaved with environment observations, ranging from 8 to 127 steps with a median of 34. All trajectories were generated with one agent configuration. This controls architectural variation within the dataset but limits generalization to other agent harnesses.

3.2 Four categories and behavioral interpretations

We classify each trajectory with deterministic rules that use tool errors, patch-application status, and overlap between edited files and the gold patch. Table 1 reports both the implemented criteria and the behavioral interpretation that motivates each scaffold family. The interpretations are hypotheses about observable trajectory structure, not independently annotated latent reasoning states.

Type	Implemented criterion	Behavioral interpretation	n	%
LOGIC	Correct file, edit applies, wrong fix	Reasoning drift	70	49
LOC	Edits wrong file or function	Mislocalized effort	37	26
EDIT	Correct location, edit command fails	Repetitive loops	28	20
PLAN	Wrong overall approach	Commitment spiral	8	6
Total			143	100

Table 1: Operational failure categories and their motivating behavioral interpretations. The rules are retrospective because file overlap uses the gold patch.

LOGIC is the largest category (49%): an edit is applied to a gold-patch file, but the task remains unresolved. We use *reasoning drift* as a behavioral interpretation for the diverse tests and revisions that can follow such an edit; the category rule itself does not establish the agent's latent reasoning process. LOC (26%) denotes either edits with no file overlap with the gold patch or repeated search without an edit. We interpret this as *mislocalized effort*, while recognizing that exploratory actions outside the gold files can still be informative.

EDIT (20%) is triggered when failed `str_replace` actions dominate the observable errors. Repeated variants of the same edit motivate the *retry-loop* interpretation and a scaffold that refreshes file context. PLAN (6%) is the residual category after the three more directly observable rules. Its *commitment* interpretation and all category-level conclusions should be treated cautiously because only eight trajectories fall in this group.

3.3 Classification methodology

The released analysis code applies a deterministic hierarchy. It assigns LOC when edited files do not overlap gold-patch files, or when repeated search produces no edit; EDIT when failed replacement operations are the dominant detected error; LOGIC when a patch was applied but the task remained unresolved; and PLAN otherwise. These rules use retrospective information and are therefore analysis labels, not an online classifier. Appendix D states the executable criteria and their limitations.

[1] `https://huggingface.co/datasets/AlexCuadron/SWE-Bench-Verified-O1-reasoning-high-results`

4 Post-Error Tail Analysis

4.1 Definition

The annotation script identifies the first *observable environment error* using a fixed set of error-message patterns. We summarize where this signal appears with the post-error tail ratio (PTR):

$$\mathrm{PTR} = \frac{n_{\text{actions at or after first observable error}}}{n_{\text{actions in the trajectory}}}. \tag{1}$$

PTR is a positional statistic, not a semantic waste annotation: actions in the tail may still gather useful evidence. It also misses silent reasoning errors that produce no recognized environment message.

4.2 Observable errors often occur early

Across the 143 failed trajectories, mean PTR is 81.8% and median PTR is 87.5%, indicating that the first recognized environment error usually occurs early in the recorded interaction. Table 2 gives the category-level breakdown.

Type	n	Mean PTR (%)	Interpretation
EDIT	28	88.8	Repetitive loops
LOGIC	70	81.2	Reasoning drift
LOC	37	80.0	Mislocalized effort
PLAN	8	70.7	Commitment spiral
All	143	81.8	—

Table 2: Post-error tail ratio by category. PTR is the fraction of recorded actions at or after the first regex-detected environment error; it does not label those actions as unproductive. The overall median is 87.5%.

PTR alone cannot distinguish repeated retries from diverse revisions. We therefore treat the structural labels in Table 2 as hypotheses motivating the controlled interventions below, rather than as conclusions of the tail statistic. The corresponding prediction is that a redirect matched to a repetitive or reversible pattern should change the proposed next action more than a redirect applied to the residual LOGIC category.

5 Behavioral Scaffolding Experiments

5.1 Setup

The released scripts construct each prompt from the first four recorded turns, not from an annotated first-error prefix. To isolate the structure–strategy interaction, they also provide the gold-derived failure category and gold target files. A model then proposes one next action under either a one-sentence scaffold or a generic retry control. This is an oracle-context diagnostic experiment, not an online detection or end-to-end repair experiment.

The automated score ranges from 0 to 3: *file hit* awards one point for mentioning a gold target-file basename, *actionable* awards one point for a regex-matched executable step, and *relevant* awards one point for category-specific recovery language. Higher is better. Because target files appear in the prompt and relevance is lexical, each component is a proxy; the score measures compliance and next-action specificity rather than patch correctness. Appendix C gives the exact rubric and an LLM-judge sensitivity check.

We test two or three scaffold candidates per category plus control. Primary results use GPT-4o-mini; a smaller common instance set is repeated across nine models in §5.4. Across matched, mismatched, and cross-model conditions, the stored outputs contain more than 1,400 scored responses. Full scaffold text appears in Appendix A.

Type	Candidate-best	Scaf.	Ctrl	Δ	95% CI	Pattern
EDIT	reread_file	2.68	1.68	**+1.00**	**[+0.71, +1.32]**	Retry loop
PLAN	step_back	2.75	1.88	**+0.88**	**[+0.50, +1.25]**	Commitment
LOC	reread_issue	1.97	1.70	+0.27	[−0.10, +0.63]	Mislocalized effort
LOGIC	minimal_fix	2.13	1.97	+0.17	[−0.07, +0.43]	Reasoning drift

Table 3: Candidate-best scaffold per category under the controlled next-action protocol (GPT-4o-mini; $n = 28, 8, 30, 30$ for EDIT, PLAN, LOC, and LOGIC, respectively). Scores range from 0 to 3; intervals are paired bootstrap 95% CIs. Because the displayed strategy is the best of two or three candidates, these comparisons are exploratory.

5.2 Category-conditioned response to scaffolds

The gains are concentrated in two categories (Table 3). For EDIT, asking the model to reread the target file raises the reported score from 1.68 to 2.68 (+1.00). This is consistent with disrupting a retry loop, but the one-step protocol cannot show that the subsequent edit succeeds. For PLAN, *step_back* raises the score by 0.88; this estimate is promising but imprecise because $n = 8$ and the category is residual.

The pattern is weaker for the other categories. The candidate-best LOC scaffold gains 0.27 and its interval crosses zero. The three LOGIC candidates cluster near control; the best reported difference is +0.17. Within this protocol, short behavioral redirects therefore change the proxy less for LOGIC than for EDIT. Richer execution evidence or multi-step verification is a plausible next test, not a conclusion of these data.

Held-out selection and evaluator sensitivity. Selecting a scaffold on nine projects and evaluating it on the held-out project retains a +0.44 full-proxy gain over control (95% project-clustered CI [+0.33,+0.67], 10 folds). Thus candidate selection on the evaluation set does not fully explain the reported full-proxy pattern. The evaluator does: across the 96 manuscript-selected pairs, the aggregate gain is +0.50 on the full proxy but 0.00 (CI [−0.09,+0.20]) after removing the type-specific relevance term; the actionable-only difference is −0.14 (CI [−0.21,+0.06]). On the 37 pairs covered by the stored blinded LLM judge, the regex difference is +0.65 but the judge difference is +0.14 (CI [−0.16,+0.24]). The large matched gains should therefore be interpreted as scaffold–evaluator alignment, not evaluator-independent next-action improvement (Figure 2a).

Mismatched strategies reduce the proxy. For LOC, *test_guided* scores 0.87 versus 1.70 for control, a 0.83-point decrease. Pairing every cross-category response with the same instance's control yields negative means for 10 of 12 off-diagonal pairs on the full proxy and 11 of 12 without the relevance term (Figure 2b; Table 11). Only three project-clustered intervals lie fully below zero, so the pair-specific estimates remain uncertain. Nevertheless, the direction is more stable than the matched gain under evaluator ablation. The result is descriptive of the tested prompt set; it does not establish that every mismatched recovery strategy is harmful.

5.3 Oracle category-conditioned selection

We compare an oracle that chooses the candidate-best scaffold using the gold-derived category, a fixed *reread_file* scaffold, and control. This analysis quantifies headroom under privileged context; it is not a deployable routing evaluation.

Condition	Score	File Hit	Score Δ
Oracle (category-specific)	2.33	83%	+0.57
Fixed (reread_file)	2.04	75%	+0.28
Control (no scaffold)	1.76	72%	—

Table 4: Oracle-context strategy comparison on 96 stored instances. The oracle uses the gold-derived category and selects the best candidate on the same evaluation set; its 2.33 score is therefore an optimistic upper bound.

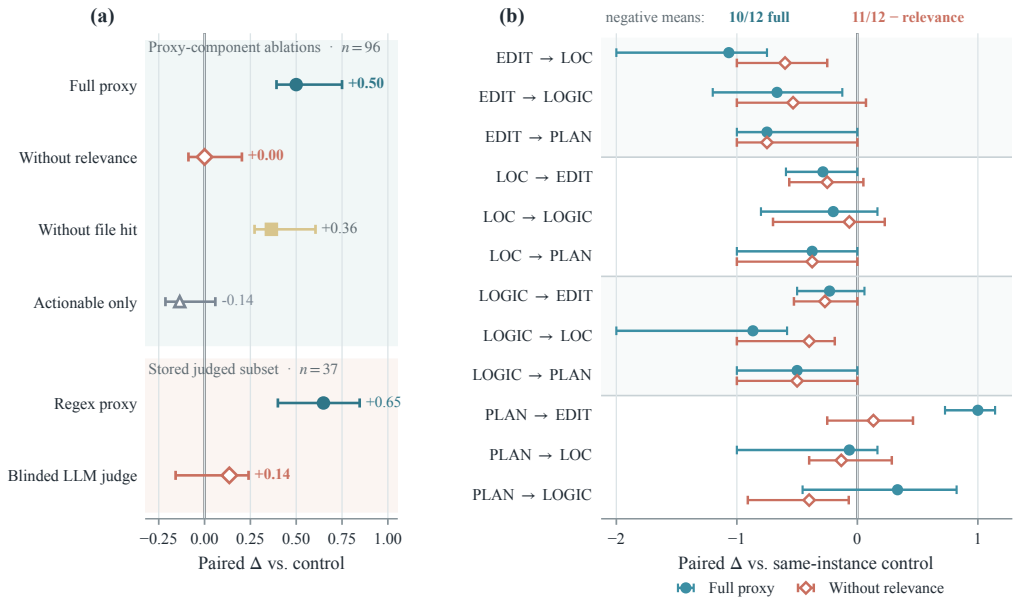

Figure 2: **Evaluator sensitivity and paired negative transfer.** (a) The matched gain is large on the full regex proxy but vanishes without its type-lexical relevance component and is small under the stored blinded judge ($n = 96$; judged subset $n = 37$). (b) Cross-category responses are paired with the same instance's control; 10 of 12 full-proxy means and 11 of 12 means without relevance are negative, although only three intervals in each specification exclude zero. Bars in both panels are 95% project-clustered bootstrap intervals.

The reported oracle score is 2.33, compared with 2.04 for the fixed scaffold and 1.76 for control. Its advantage is partly selected on the evaluation set and thus should be read as evidence that scaffold effects differ by category, not as an unbiased estimate of a learned router. A deployable test requires a strictly observable classifier, held-out strategy selection, and prompts that omit gold category and target files.

5.4 Cross-Model Analysis

We repeat one matched scaffold and control on a common subset for nine models from four families. This checks whether the small LOGIC effect is specific to GPT-4o-mini, while retaining all limitations of the proxy protocol. Figure 3 visualizes the differences and Table 5 reports the full matrix.

		EDIT		LOC		LOGIC		PLAN	
Model	**Tier**	**Ctrl**	**Δ**	**Ctrl**	**Δ**	**Ctrl**	**Δ**	**Ctrl**	**Δ**
GPT-4o-mini	mid	1.61	+1.04	1.54	+0.49	2.01	+0.13	1.88	+0.88
DS-V4-Flash	mid	2.27	+0.60	2.00	+0.40	2.33	+0.13	1.88	+0.63
Qwen 3.5	mid	2.40	+0.00	2.13	+0.13	2.33	+0.07	1.62	+1.38
DS-V4-Pro	strong	1.53	+0.73	1.67	+0.87	2.33	+0.07	1.88	−0.38
GPT-4.1	strong	1.80	+0.60	2.27	−0.20	2.33	0.00	1.88	−0.38
Claude Son. 4.6	strong	2.79	−0.07	2.50	+0.13	2.50	0.00	2.00	+0.12
Claude Opus 4.7	frontier	2.27	+0.13	2.47	+0.27	2.40	+0.13	2.00	0.00
o4-mini†	frontier	1.53	+0.53	0.93	+1.47	1.33	−0.20	1.75	+0.38
GPT-5.5†	frontier	1.00	+1.20	0.80	+1.60	1.60	−0.40	1.60	−0.20

Table 5: Scaffold effect (Δ) across nine models under the same oracle-context next-action protocol. †Reasoning-chain models. The LOGIC column ranges from −0.40 to +0.13; other categories show mixed, model-dependent effects.

Figure 3: **Cross-model scaffold effects.** The heatmap uses oracle diagnostic context. The continuous diverging scale is centered at zero, and horizontal rules separate the mid, strong, and frontier tiers in Table 5. The LOGIC differences are small or negative for all nine models; effects in the other columns vary across models.

The LOGIC effect remains small or negative for every model (range −0.40 to +0.13), making this the most consistent cross-model observation. EDIT is positive for seven of nine models, but the magnitude varies from −0.07 to +1.20. LOC and PLAN are less stable: both contain negative cells and large positive cells. These mixed results argue against a universal redirect. They also show that the current study cannot separate category structure from model-specific prompt sensitivity. Because this matrix uses the full regex proxy, it should be read as a cross-model prompt compliance diagnostic rather than independent validation of recovery quality.

6 Discussion

The controlled results support a cautious control hypothesis. A redirect can strongly change lexical and target-following behavior, but the matched advantage does not survive removal of the relevance component or the blinded-judge check. By contrast, negative off-diagonal transfer remains common after the relevance ablation. This asymmetry suggests that a controller should validate an intervention and be allowed to abstain rather than always emit a redirect.

The evidence does not yet justify a deployed recovery architecture. The category and target files are privileged inputs, the best scaffold is selected on the evaluated categories, and the score does not verify a patch. A stronger test would detect structure from pre-intervention observations, select a strategy on held-out data, and measure whether the redirected agent subsequently passes repository tests. Under that design, the present results provide hypotheses and candidate interventions rather than performance guarantees.

7 Conclusion

In 143 failed code-agent trajectories, observable environment errors tend to occur early and scaffold effects differ by operational category on a full regex proxy. The apparent matched gain is not evaluator-independent: aggregate Δ changes from +0.50 to 0.00 without type-lexical relevance, and a blinded judge gives +0.14 on the available paired subset. Negative transfer is more persistent, with 10 of 12 paired means below control on the full proxy and 11 of 12 below control without relevance. These results motivate conservative intervention with abstention, not a claim of repair. Actual recovery requires removing privileged inputs and measuring multi-step repository resolution.

Limitations

The dataset contains failed trajectories from one agent configuration on Python repositories in SWE-bench Verified, so category prevalence need not transfer to other harnesses, languages, or successful runs. Categories use gold-patch overlap retrospectively, and PLAN is a small residual group ($n = 8$). The behavioral interpretations are not independently annotated structural measurements.

The intervention prompts contain the gold-derived category and target files and use the first four recorded turns rather than a synchronized first-error prefix. The outcome is one generated action scored by lexical proxies; target-file hit is especially easy because the target is disclosed. The experiment therefore does not measure repair success, cost, or multi-turn side effects. Candidate-best strategies are selected from two or three prompts on the evaluation categories, although leave-one-project-out selection retains a full-proxy gain. More importantly, the matched gain vanishes when the type-lexical relevance component is removed and is small on the incomplete blinded-judge subset. Cross-model results reuse the full proxy and should not be interpreted as independent end-to-end validation.

Selective intervention could reduce unnecessary model calls and developer waiting time if validated in a real agent loop. Conversely, a mistimed controller could suppress useful exploration or steer an agent toward an incorrect repository change. The present proxy study is not sufficient for deployment in safety-critical software workflows.

References

Islem Bouzenia, Yangruibo Ding, Kexin Pei, Baishakhi Ray, and Michael Pradel. TraceFixer: Execution trace-driven program repair. *arXiv preprint arXiv:2304.12743*, 2023.

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya G. Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. CodeR: Issue resolving with multi-agent and task graphs. *arXiv preprint arXiv:2406.01304*, 2024.

Xiancai Chen, Zhengwei Tao, Kechi Zhang, Changzhi Zhou, Xinyu Zhang, Wanli Gu, Yuanpeng He, Mengdi Zhang, Xunliang Cai, Haiyan Zhao, and Zhi Jin. Revisit self-debugging with self-generated tests for code generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

Xinyun Chen, Maxwell Lin, Nathanael Sch"arli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.

Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for LLM agent training. *arXiv preprint arXiv:2505.10978*, 2025.

Shubham Gandhi, Jason Tsay, Jatin Ganhotra, Kiran Kate, and Yara Rizk. When agents go astray: Course-correcting SWE agents with PRMs. *arXiv preprint arXiv:2509.02360*, 2025.

Dong Huang, Jie Bu, Jie M. Zhang, Michael Luck, and Cong Wen. AgentCoder: Multi-agent code generation with effective testing and self-optimisation. *arXiv preprint arXiv:2312.13010*, 2024.

Jiangping Huang, Wenguang Ye, Weisong Sun, Jian Zhang, Mingyue Zhang, and Yang Liu. TraceCoder: A trace-driven multi-agent framework for automated debugging of LLM-generated code. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026.

Zhili Huang, Ling Xu, Chao Liu, Weifeng Sun, Xu Zhang, Yan Lei, Meng Yan, and Hongyu Zhang. DynaFix: Iterative automated program repair driven by execution-level dynamic information. *arXiv preprint arXiv:2512.24635*, 2025.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. Using an LLM to help with code understanding. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*, 2024.

Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

Shahd Seddik and Fatemeh Fard. ARISE: A repository-level graph representation and toolset for agentic fault localization and program repair. *arXiv preprint arXiv:2605.03117*, 2026.

Melika Sepidband, Hamed Taherkhani, Hung Viet Pham, and Hadi Hemmati. RGFL: Reasoning guided fault localization for automated program repair using large language models. *arXiv preprint arXiv:2601.18044*, 2026.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. MAGIS: LLM-based multi-agent framework for GitHub issue resolution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the ACM on Software Engineering (FSE)*, 2025.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Siyu Yuan, Zehui Chen, Zhiheng Xi, Junjie Ye, Zhengyin Du, and Jiecao Chen. Agent-R: Training language model agents to reflect via iterative self-training. *arXiv preprint arXiv:2501.11425*, 2025.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V. Le, and Denny Zhou. Take a step back: Evoking reasoning via abstraction in large language models. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, and Jiaxun Zhang. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*, 2025.

A Prompt Templates

All scaffold prompts are short directives appended after the first four recorded turns. The surrounding template states the gold-derived failure category and gold target files. The control receives a generic retry instruction ("try again with a different approach") under the same diagnostic context. Below we reproduce the scaffold-specific sentence.

A.1 EDIT Strategies

reread_file: "Before attempting the edit again, re-read the target file to refresh your understanding of its current contents."

smaller_edit: "Try breaking your edit into smaller, more targeted changes rather than replacing a large block at once."

A.2 PLAN Strategies

step_back: "Step back and re-read the original issue description. Reconsider whether your current approach is the right way to address what is being asked."

scope_check: "Verify that the scope of your current changes matches what the issue is actually requesting."

A.3 LOC Strategies

reread_issue: "Re-read the original issue description carefully and identify any clues about which specific file or function is responsible for the reported behavior."

test_guided: "Run the relevant test suite to identify which component is actually failing, and use that to guide your search for the right file to modify."

broaden_search: "Stop working on your current target file. Search more broadly. Check related modules, imported files, and parent classes. Trace the symptoms to a different code location."

A.4 LOGIC Strategies

minimal_fix: "Focus on making the smallest possible change that addresses the core issue, rather than a comprehensive rewrite."

test_analysis: "Analyze the failing test assertion carefully. What exact value or behavior does it expect? Compare against what your code produces, and identify the specific computation your fix gets wrong."

edge_cases: "Your fix may address the common case but miss edge cases that the test exercises. Consider boundary conditions (empty inputs, zero, negative values) and re-read the test to identify which case it targets."

B Detailed Strategy Results

Tables 6–9 report aggregate score components for all strategies and the control condition.

Metric	reread	smaller	control
Mean Score	2.68	1.86	1.68
File Hit (%)	89	89	75
Actionable (%)	100	96	89
Relevant (%)	79	0	4

Table 6: Strategy comparison for EDIT failures ($n = 28$, GPT-4o-mini).

Metric	step_back	scope	control
Mean Score	2.75	1.88	1.88
File Hit (%)	100	88	88
Actionable (%)	88	100	100
Relevant (%)	88	0	0

Table 7: Strategy comparison for PLAN failures ($n = 8$, GPT-4o-mini).

Metric	reread	test_g.	broaden	ctrl
Mean Score	1.97	0.87	1.93	1.70
File Hit (%)	73	23	87	53
Actionable (%)	23	63	70	83
Relevant (%)	100	0	37	33

Table 8: Strategy comparison for LOC failures ($n = 30$, GPT-4o-mini). test_g. denotes test_guided, which scores 0.83 below control.

Metric	min_fix	test_a.	edge_c.	ctrl
Mean Score	2.13	2.07	2.07	1.97
File Hit (%)	90	87	90	83
Actionable (%)	100	80	83	90
Relevant (%)	23	40	33	23

Table 9: Strategy comparison for LOGIC failures ($n = 30$, GPT-4o-mini). All strategy means lie within 0.16 points of one another.

C Statistical Methods

Bootstrap confidence intervals. For Table 3, we align scaffold and control scores by instance, resample the paired differences with replacement 10,000 times, and report the 2.5th and 97.5th percentiles. The local audit script uses a fixed random seed and reproduces the four displayed intervals.

Selection and multiple comparisons. The strategy shown for each category is the largest observed mean among two or three candidates, and the oracle reuses this selection. The bootstrap intervals condition on the selected candidate and are not adjusted for selection or multiple comparisons. We therefore treat these results as exploratory and do not make statistical-significance claims from them.

Project-held-out selection. We group instances by their source project. For each of 10 folds, candidate means are computed on the other nine projects, one scaffold is selected per category, and its stored response is evaluated on the held-out project. The resulting 96 paired differences are summarized with a project-clustered percentile bootstrap (10,000 resamples, fixed seed). This yields 2.229 for the selected scaffold versus 1.792 for control, $\Delta = +0.438$ with 95% CI [+0.333,+0.674].

Proxy ablations. Using the same manuscript-selected scaffold for each category, the aggregate paired difference is +0.500 [+0.393,+0.750] on the full proxy, 0.000 [-0.088,+0.204] without type-specific relevance, +0.365 [+0.273,+0.605] without file hit, and -0.135 [-0.213,+0.059] for actionability alone. Intervals resample source projects and therefore reflect cross-project rather than generation-seed variability.

Effect sizes. All reported Δ values are paired mean differences on the 0–3 scoring scale. We report this native unit rather than a standardized effect because the three-point total is discrete and its components are heterogeneous.

Automated scoring rubric. Scoring is automated via pattern matching on model outputs. *File hit* (1 point): the response references the correct target file (identified by basename matching against the reference solution). *Actionable* (1 point): the response matches a regex for executable commands or code edits (e.g., `cat`, `grep`, `python`, `str_replace`). *Relevant* (1 point): type-specific regex detects appropriate recovery behavior, e.g., for EDIT, terms like "read," "view," "exact," "whitespace"; for PLAN, terms like "approach," "alternative," "reconsider." The rubric is reproducible but lexically biased: a response can earn relevance by echoing scaffold vocabulary without improving a patch. We therefore compare it with two stored LLM-judge evaluations; these are sensitivity analyses, not human validation. First, GPT-4.1 judged 60 sampled responses (15 per type) with access to gold files and failure-type context; within-$\pm$1 agreement was 95% (57/60). Second, we expanded to 220 responses using a *blinded* GPT-4o-mini judge that received no type-specific keywords in its rubric, only "file relevant," "actionable," and "progress" dimensions. On this blinded evaluation, within-$\pm$1 agreement is 82%. Only 37 responses form matched scaffold–control pairs for the manuscript-selected strategies. On this paired subset, the regex difference is +0.649 [+0.400,+0.846], whereas the blinded-judge difference is +0.135 [-0.158,+0.240]. This discrepancy reinforces the need to interpret the full proxy as lexical compliance rather than evaluator-independent action quality.

File-hit sensitivity analysis. Removing the type-specific relevance component leaves a simpler binary outcome: whether the response mentions a gold target-file basename. Table 10 shows that the direction of several effects remains visible. However, this is not an independent correctness measure because the prompt itself discloses the target files; it measures whether the model follows that supplied context.

Type	Strategy	Hit	Ctrl	Δ (pp)	p
EDIT	reread_file	89%	75%	+14	0.30
PLAN	step_back	100%	88%	+13	1.00
LOC	broaden	87%	53%	+33	0.01*
LOGIC	minimal_fix	90%	83%	+7	0.71
LOC	test_guided	23%	53%	−30	0.03*

Table 10: Target-file mention rates by category and strategy. The outcome has no type-specific relevance regex, but the gold target files are supplied in the prompt, so it should be interpreted as context-following rather than repair correctness. Reported Fisher tests are uncorrected descriptive analyses.

Cross-model experiment details. All nine models were evaluated on the same instance set (15 per type for EDIT/LOC/LOGIC, 8 for PLAN), yielding 53 scored responses per model per condition. API calls used temperature 0 for standard models and default settings for reasoning-chain models (†), accessed via a unified API proxy between April–May 2026. All models received the same prompt construction, including gold-derived category and target-file context.

D Classification Methodology Details

D.1 Hierarchical Classification Rules

The released annotation script proceeds through the following decision tree:

1. **Localization**: assign LOC if at least one edit occurs but no edited path overlaps a gold-patch path, or if the trajectory contains no edit and at least three search actions.

2. **Edit application**: assign EDIT after at least two recognized replacement errors, or after one replacement error when no patch was applied and replacement errors constitute at least half of detected errors.

3. **Applied patch**: assign LOGIC when the harness records that a patch was applied but the instance remains failed.

4. **Residual rules**: the code next checks repeated test errors, long correct-file trajectories, and search-heavy behavior; remaining cases are assigned PLAN. No trajectory in the analyzed file receives the intermediate TEST label.

The hierarchy is intentionally pragmatic rather than ontological. In particular, PLAN is a fallback, and both file overlap and patch status depend on retrospective benchmark artifacts. The repository does not contain the manual-label file described in an earlier draft, so we do not report an inter-annotator agreement claim. Likewise, stored online-classifier summaries are internally inconsistent with the draft confusion matrix and lack the training script and per-instance predictions; online routing results are omitted from this version.

E Paired Cross-Category Transfer

Table 11 reports 183 stored cross-category evaluations. Each response is paired with its own instance's control, avoiding the earlier comparison between subset scaffold means and category-wide control means. Ten of twelve full-proxy means are negative; 11 of 12 are negative when the type-lexical relevance component is removed. Only three intervals in each specification lie fully below zero.

Source	Target	n	Full Δ [95% CI]	No-rel. Δ [95% CI]
EDIT	LOC	15	−1.07 [−2.00,−0.75]	−0.60 [−1.00,−0.25]
EDIT	LOGIC	15	−0.67 [−1.20,−0.13]	−0.53 [−1.00,+0.07]
EDIT	PLAN	8	−0.75 [−1.00,0.00]	−0.75 [−1.00,0.00]
LOC	EDIT	28	−0.29 [−0.59,0.00]	−0.25 [−0.57,+0.05]
LOC	LOGIC	15	−0.20 [−0.80,+0.17]	−0.07 [−0.70,+0.23]
LOC	PLAN	8	−0.38 [−1.00,0.00]	−0.38 [−1.00,0.00]
LOGIC	EDIT	26	−0.23 [−0.50,+0.06]	−0.27 [−0.53,0.00]
LOGIC	LOC	15	−0.87 [−2.00,−0.58]	−0.40 [−1.00,−0.19]
LOGIC	PLAN	8	−0.50 [−1.00,0.00]	−0.50 [−1.00,0.00]
PLAN	EDIT	15	+1.00 [+0.73,+1.14]	+0.13 [−0.25,+0.46]
PLAN	LOC	15	−0.07 [−1.00,+0.17]	−0.13 [−0.40,+0.29]
PLAN	LOGIC	15	+0.33 [−0.45,+0.82]	−0.40 [−0.91,−0.07]

Table 11: Paired off-diagonal transfer. No-rel. removes the type-specific relevance point from both scaffold and control scores. Intervals are 95% project-clustered percentile-bootstrap intervals with 10,000 resamples.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Sections 1 and 7 explicitly limit the evidence to an oracle-context, one-step next-action proxy and distinguish it from end-to-end repair.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The dedicated Limitations section discusses the single harness, retrospective labels, privileged prompt context, proxy outcome, strategy selection, and small Plan category.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.

- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper contains no theoretical results or proofs.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 5 and Appendices A–D specify prompt construction, scoring, sample sizes, category rules, and bootstrap computation; the supplementary artifact contains stored responses and a local audit script.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case

of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: The underlying public trajectory dataset is linked, and code/results are prepared as supplementary material, but no permanent anonymous public code URL is provided in this draft.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 5 defines the evaluation contract and Appendix C reports sample sizes, model settings, prompt context, scoring rules, and statistical procedures.

   Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Table 3 reports paired percentile-bootstrap 95% confidence intervals, and Appendix C states the resampling unit, number of resamples, and selection caveat.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The stored artifacts record API model settings but do not preserve reliable wall-clock time, monetary cost, or provider hardware for the original generations.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [N/A]

Justification: This draft does not record an author review of the NeurIPS Code of Ethics; this item should be confirmed before submission.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Limitations section notes both the potential compute benefit of selective intervention and the risk that a mistimed controller suppresses useful exploration.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The work releases neither a new pretrained model nor a high-risk dataset requiring access safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.

- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [No]

    Justification: Existing datasets and models are cited and linked, but this draft does not yet provide a complete license inventory for every upstream asset.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: The paper does not introduce a new dataset or pretrained model as a contribution.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The study uses public software-agent trajectories and model outputs; it includes no crowdsourcing or human-subject experiment.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human-subject research was conducted.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: LLMs are the evaluated systems and are also used in the appendix sensitivity checks; their roles are described in Sections 5 and C.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.